JOBSHEET 9

Array 1 Dimensi

1. Tujuan

- Mahasiswa mampu memahami konsep array 1 dimensi serta cara mendeklarasikan, menginisialisasi, dan mengakses elemennya dalam bahasa pemrograman Java.
- Mahasiswa mampu membuat program yang memanfaatkan array 1 dimensi untuk menyimpan dan mengolah data, termasuk menggunakan perulangan for dan foreach.
- Mahasiswa mampu menerapkan operasi aritmatika pada elemen array, seperti menghitung total, rata-rata, nilai tertinggi, dan nilai terendah.
- Mahasiswa mampu menerapkan algoritma pencarian (searching) dan pengurutan (sorting) sederhana pada array 1 dimensi.
- Mahasiswa mampu menerapkan array 1 dimensi untuk menyelesaikan studi kasus yang relevan dengan kehidupan kampus dan sehari-hari.

2. Dasar Teori

2.1. Apa itu Array?

Array adalah struktur data yang menyimpan sekumpulan data bertipe sama secara berurutan dalam satu variabel. Alih-alih membuat variabel terpisah untuk setiap data (misalnya nilai1, nilai2, nilai3, ... nilai10), array memungkinkan seluruh data tersebut disimpan dalam satu variabel saja dan diakses menggunakan indeks. Pada Java, array bersifat homogen (seluruh elemen harus bertipe sama) dan berukuran tetap (fixed) — ukurannya tidak dapat diperbesar atau diperkecil setelah array diinstansiasi.

Secara internal, elemen-elemen array disimpan secara berurutan (contiguous) di dalam memori, sehingga komputer dapat mengakses elemen mana pun secara langsung berdasarkan indeksnya tanpa perlu menelusuri elemen-elemen sebelumnya satu per satu. Inilah salah satu alasan mengapa mengakses elemen array (misalnya nilai[5]) berlangsung sangat cepat dibandingkan struktur data lain yang tidak tersimpan berurutan.

2.2. Deklarasi dan Inisialisasi Array

Terdapat beberapa cara untuk mendeklarasikan dan menginisialisasi array 1 dimensi pada Java, tergantung apakah nilai elemen sudah diketahui sejak awal atau akan diisi kemudian.

Sintaks	Penjelasan
`int[] bil = new int[4];`	Deklarasi array kosong berkapasitas 4 elemen. Seluruh elemen otomatis terisi nilai default (0 untuk tipe int) hingga diisi secara eksplisit.
`int[] bil = {5, 13, -7, 17};`	Deklarasi sekaligus inisialisasi nilai elemen secara langsung. Ukuran array otomatis mengikuti banyaknya nilai yang dituliskan.

`int[] bil = new int[]{5, 13, -7, 17};`	Bentuk lain dari inisialisasi langsung, umum dipakai ketika array dideklarasikan sebagai argumen method atau nilai balik (return value).
`String[] menu = {"Nasi Goreng", "Teh Tarik"};`	Array juga dapat menyimpan tipe data non-primitif seperti String, sepanjang seluruh elemennya bertipe sama.

2.3. Indeks, Panjang Array, dan Kesalahan Umum

Indeks array pada Java selalu dimulai dari 0, sehingga elemen pertama memiliki indeks 0 dan elemen terakhir memiliki indeks (panjang array − 1). Panjang atau kapasitas suatu array dapat diketahui melalui atribut .length, misalnya bil.length. Perlu diperhatikan bahwa .length adalah atribut (bukan method), sehingga ditulis tanpa tanda kurung — berbeda dengan String yang menggunakan method .length().

Kesalahan yang paling sering terjadi bagi pemula adalah mencoba mengakses indeks yang berada di luar rentang array, misalnya mengakses bil[4] pada array berkapasitas 4 elemen (indeks valid hanya 0 sampai 3). Kesalahan ini akan menyebabkan program berhenti dengan error ArrayIndexOutOfBoundsException pada saat program dijalankan (runtime error), bukan pada saat program di-compile.

Nilai Default	Berlaku untuk Tipe
`0 (nol)`	Nilai default elemen array bertipe int atau double sebelum diisi secara eksplisit.
`false`	Nilai default elemen array bertipe boolean.
`null`	Nilai default elemen array bertipe objek/referensi, misalnya String.

2.4. Mengolah Array dengan Perulangan

Ada beberapa cara untuk menelusuri (iterate) seluruh elemen array pada Java, masing-masing dengan kegunaannya sendiri.

Teknik	Kegunaan
`for (int i = 0; i < arr.length; i++)`	Cara paling fleksibel — indeks i dapat dimanfaatkan untuk mengakses, mengubah nilai elemen, atau membandingkan elemen dengan elemen lain (misalnya arr[i] dan arr[i+1]).
`for (int nilai : arr)`	Foreach loop (enhanced for-loop) — lebih ringkas untuk sekadar membaca/menampilkan setiap elemen, namun tidak dapat mengetahui indeks elemen tersebut secara langsung dan tidak dapat mengubah isi array.
`Arrays.toString(arr)`	Method bantu dari kelas java.util.Arrays untuk menampilkan seluruh isi array dalam satu baris dengan format [elemen1, elemen2, ...], tanpa perlu menulis perulangan manual.
`Arrays.sort(arr)`	Method bantu untuk mengurutkan array secara ascending (menaik) menggunakan algoritma sorting yang sudah teroptimasi secara internal.

2.5. Operasi Umum pada Array: Agregasi, Pencarian, dan Pengurutan

Tiga jenis operasi yang paling sering diterapkan pada array adalah:

- Agregasi (total, rata-rata, nilai maksimum/minimum) — dilakukan dengan menelusuri seluruh elemen sambil mengakumulasikan hasil ke dalam satu variabel (misalnya total) atau membandingkannya dengan variabel penampung sementara (misalnya tertinggi).
- Pencarian (searching) — menemukan apakah suatu nilai ada di dalam array, dan pada indeks berapa nilai tersebut berada. Teknik paling sederhana adalah linear search, yaitu memeriksa elemen satu per satu dari awal hingga ditemukan atau seluruh elemen habis diperiksa.
- Pengurutan (sorting) — menyusun ulang elemen array berdasarkan urutan tertentu (menaik/ascending atau menurun/descending). Salah satu algoritma sorting paling sederhana untuk dipelajari adalah bubble sort, yang bekerja dengan membandingkan dan menukar posisi dua elemen bersebelahan secara berulang.

Sebagai gambaran tingkat efisiensi (bukan untuk dihafal, cukup untuk membangun intuisi), berikut adalah kompleksitas waktu masing-masing teknik pada kondisi terburuk (worst case) menggunakan notasi Big-O — semakin besar n (jumlah elemen array), semakin terasa perbedaan efisiensinya:

Operasi	Kompleksitas Waktu (Big-O)
`Mengakses elemen berdasarkan indeks (arr[i])`	O(1) — waktu konstan, secepat apa pun ukuran array.
`Linear search`	O(n) — waktu bertambah sebanding lurus dengan jumlah elemen, karena elemen diperiksa satu per satu.
`Bubble sort`	$O(n^2)$ — waktu bertambah secara kuadratik terhadap jumlah elemen, karena terdapat dua perulangan bersarang (nested loop).

2.6. Array vs ArrayList: Kapan Menggunakan yang Mana?

Java juga menyediakan struktur data ArrayList (bagian dari Java Collection Framework) yang serupa dengan array, namun ukurannya dapat bertambah atau berkurang secara dinamis mengikuti kebutuhan program saat dijalankan.

Aspek	Perbandingan
`Ukuran`	Array bersifat tetap (fixed) sejak diinstansiasi. ArrayList dapat bertambah/berkurang secara dinamis.
`Tipe elemen`	Array dapat menampung tipe primitif (int, double, dll) secara langsung. ArrayList hanya dapat menampung tipe objek (harus menggunakan Integer, Double, dll).
`Performa`	Array umumnya sedikit lebih cepat dan hemat memori karena strukturnya lebih sederhana.
`Kemudahan penggunaan`	ArrayList menyediakan banyak method siap pakai (add, remove, contains, dll) yang memudahkan pengolahan data dibandingkan array.

Sebagai aturan umum: gunakan array apabila jumlah data sudah diketahui dan tidak akan berubah selama program berjalan (misalnya jumlah hari dalam seminggu, atau jumlah menu yang tetap). Gunakan ArrayList

apabila jumlah data bisa bertambah/berkurang secara dinamis (misalnya daftar pesanan yang terus bertambah). Materi ArrayList akan dibahas lebih lanjut pada jobsheet berikutnya.

2.7. Relevansi Array 1 Dimensi pada Studi Kasus Praktikum

Pada praktikum ini, array 1 dimensi akan diterapkan pada studi kasus yang relevan dengan lingkungan kampus, yaitu pengelolaan nilai mahasiswa pada Sistem Informasi Akademik (SIAKAD) — mencakup input nilai, penentuan status kelulusan, hingga perhitungan statistik seperti rata-rata dan predikat kelas. Selain itu, array 1 dimensi juga diterapkan pada studi kasus pengelolaan pemesanan pada kafe kampus, mencakup pencatatan daftar menu, pencarian menu, pencatatan pesanan, dan perhitungan total biaya.

Kedua studi kasus tersebut merepresentasikan pola penggunaan array yang sangat umum dijumpai pada aplikasi nyata: menyimpan sekumpulan data sejenis (nilai mahasiswa, daftar menu, daftar harga), lalu mengolahnya menggunakan kombinasi perulangan, agregasi, pencarian, dan pengurutan seperti yang telah dijelaskan pada bagian-bagian sebelumnya.

Menjalankan Program di VS Code: Seluruh kode program pada jobsheet ini ditulis dan dijalankan menggunakan Visual Studio Code. Buat file baru sesuai nama yang diminta pada tiap langkah (File > New File, lalu simpan/Save As dengan nama dan ekstensi .java yang sesuai). Untuk meng-compile dan menjalankan program, buka terminal terintegrasi VS Code melalui menu Terminal > New Terminal atau shortcut Ctrl + ` (backtick), lalu jalankan perintah javac NamaFile.java untuk meng-compile, dilanjutkan java NamaFile (tanpa .java) untuk menjalankan program.

3. Praktikum

3.1. Percobaan 1: Deklarasi, Inisialisasi, Mengisi, dan Menampilkan Elemen Array

Waktu Percobaan: 30 menit

1. Buka VS Code. Buat file baru (**File > New File**), lalu simpan dengan nama **ArrayBilangan<NoAbsen>.java**
2. Buat array bertipe integer dengan nama bil berkapasitas 4 elemen:

```
int[] bil = new int[4];
```

3. Isi masing-masing elemen array bil dengan angka 5, 13, -7, 17:

```
bil[0] = 5;
bil[1] = 13;
bil[2] = -7;
bil[3] = 17;
```

4. Tampilkan seluruh isi elemennya ke layar menggunakan perulangan for:

```
for (int i = 0; i < bil.length; i++) {
    System.out.println("bil[" + i + "] = " + bil[i]);
}
```

5. Sebagai pengayaan, tampilkan juga isi array menggunakan foreach loop, dan bandingkan dengan menampilkan array secara langsung memakai method bantu Arrays.toString():

```
for (int nilai : bil) {
    System.out.println(nilai);
}

import java.util.Arrays;
System.out.println(Arrays.toString(bil));
```

6. Buka terminal VS Code (Ctrl + `), lalu compile dan jalankan program dengan perintah: javac ArrayBilangan<NoAbsen>.java lalu java ArrayBilangan<NoAbsen>. Amati dan cocokkan hasilnya.

Contoh Hasil Percobaan 1 (Acuan)



Contoh tampilan terminal VS Code setelah program dijalankan. Hasil pada laptop masing-masing mahasiswa harus sesuai dengan tampilan di atas.

Pertanyaan Percobaan 1

1. Jika isi masing-masing elemen array bil diubah dengan angka 5.0, 12867, 7.5, 2000000, apa yang terjadi? Mengapa bisa demikian?
2. Modifikasi kode program di atas dengan melakukan inisialisasi elemen array sekaligus pada saat deklarasi array (tanpa new int[4] dan pengisian satu per satu).
3. Ubah statement penampilan pada langkah ke-4 menjadi seperti berikut: System.out.println("bil[" + i + "] = " + bil[i] + ", double = " + (bil[i] * 2)); Apa keluaran dari program? Jelaskan maksud dari statement tersebut.
4. Jika kondisi pada statement for-loop di atas diubah menjadi i <= 4, apa keluaran dari program? Mengapa demikian?
5. Apa perbedaan antara menampilkan array menggunakan for loop, foreach loop, dan Arrays.toString() pada langkah ke-5? Kapan sebaiknya masing-masing digunakan?

3.2. Percobaan 2: Studi Kasus SIAKAD — Input Nilai Mahasiswa dan Status Kelulusan

Waktu Percobaan: 45 menit

Pada Sistem Informasi Akademik (SIAKAD), dosen perlu menginput nilai akhir mahasiswa dan mengetahui status kelulusan masing-masing mahasiswa secara cepat.

1. Buka VS Code. Buat file baru, simpan dengan nama **ArrayNilai<NoAbsen>.java**
2. Buatlah struktur dasar Java (class dan method main), lalu import library Scanner.
3. Buat deklarasi array bertipe integer dengan nama nilaiAkhir berkapasitas 10 elemen:

```
int[] nilaiAkhir = new int[10];
Scanner input = new Scanner(System.in);
```

4. Buat struktur perulangan untuk menerima input dan mengisi elemen array nilaiAkhir:

```
for (int i = 0; i < nilaiAkhir.length; i++) {
    System.out.print("Masukkan nilai mahasiswa ke-" + (i + 1) + ": ");
    nilaiAkhir[i] = input.nextInt();
}
```

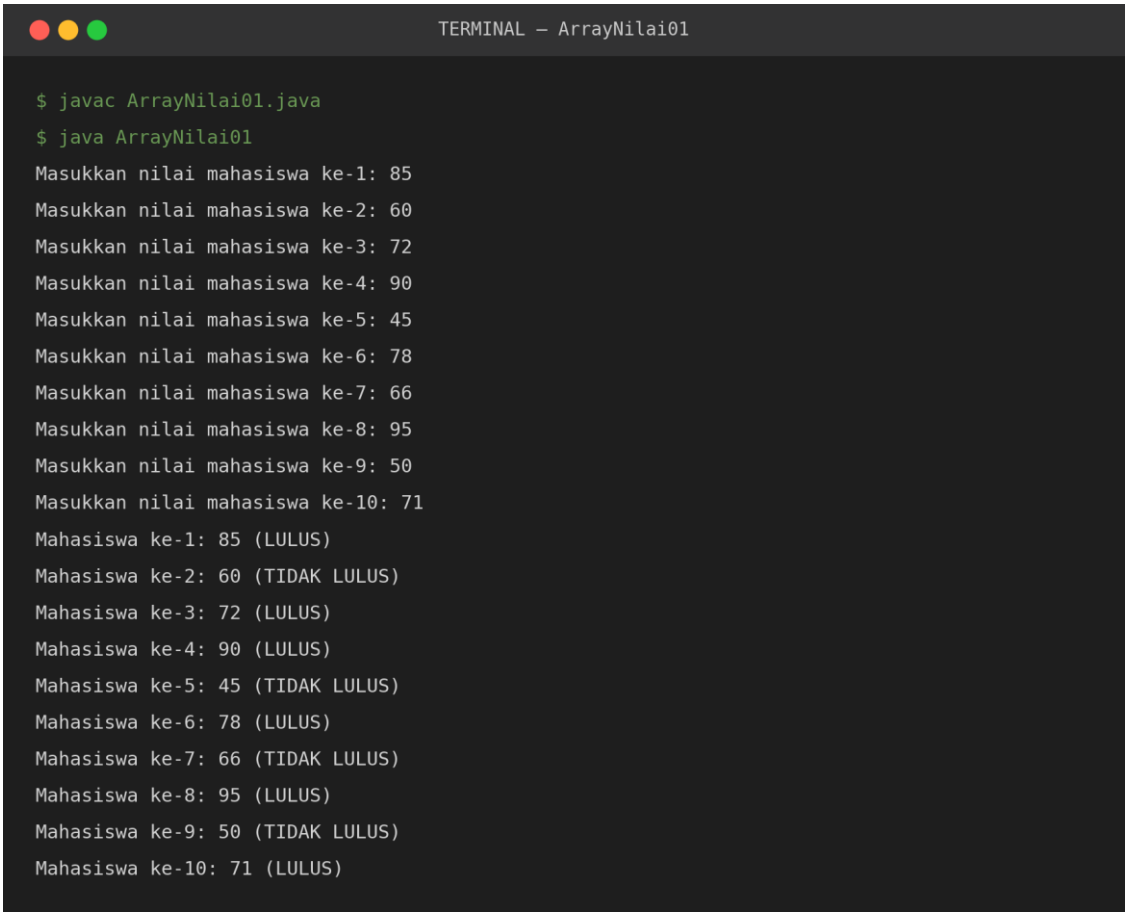
5. Menggunakan struktur perulangan, tampilkan seluruh isi elemen array nilaiAkhir beserta status kelulusannya (LULUS apabila nilai lebih dari 70, TIDAK LULUS apabila sebaliknya):

```
for (int i = 0; i < nilaiAkhir.length; i++) {
    String status = (nilaiAkhir[i] > 70) ? "LULUS" : "TIDAK LULUS";
    System.out.println("Mahasiswa ke-" + (i + 1) + ": " + nilaiAkhir[i] + " (" +
status + ")");
}
```

6. Buka terminal VS Code, compile dan jalankan program dengan perintah: javac **ArrayNilai<NoAbsen>.java** lalu java **ArrayNilai<NoAbsen>**. Amati hasilnya dengan menginput beberapa variasi nilai.

Contoh Hasil Percobaan 2 (Acuan)

Berikut contoh hasil apabila program dijalankan dengan input data nilai: 85, 60, 72, 90, 45, 78, 66, 95, 50, 71 (mahasiswa dapat menggunakan data nilai lain, namun pola tampilan output harus mengikuti format yang sama).



Contoh tampilan terminal VS Code setelah program dijalankan dengan data contoh di atas.

Pertanyaan Percobaan 2

1. Apa yang dimaksud dengan kondisi i < nilaiAkhir.length pada perulangan di atas?
2. Modifikasi program agar hanya menampilkan nilai mahasiswa yang lulus saja (nilai > 70), tanpa menampilkan yang tidak lulus. Jelaskan alur programnya!
3. Ubah perulangan pada langkah ke-4 dan ke-5 dari for loop menjadi foreach loop. Apakah seluruh bagian program dapat diubah menjadi foreach? Jelaskan bagian mana yang bisa dan tidak bisa, beserta alasannya.
4. Modifikasi program agar turut menghitung dan menampilkan jumlah mahasiswa yang lulus dan jumlah mahasiswa yang tidak lulus di akhir program.

3.3. Percobaan 3: Operasi Aritmatika pada Array — Total, Rata-rata, Nilai Tertinggi dan Terendah

Waktu Percobaan: 45 menit

Percobaan ini melanjutkan studi kasus SIAKAD untuk menghasilkan ringkasan nilai mahasiswa, sebagaimana biasa ditampilkan pada rekap nilai akhir semester.

1. Buka VS Code. Buat file baru, simpan dengan nama **ArrayRataNilai<NoAbsen>.java**
2. Buatlah struktur dasar Java, import dan deklarasikan Scanner untuk keperluan input.
3. Buat array nilaiMhs bertipe integer dengan kapasitas 10, serta deklarasikan variabel total dan rata2:

```
int[] nilaiMhs = new int[10];
int total = 0;
double rata2;
```

4. Isi array nilaiMhs dengan nilai dari input pengguna, sekaligus akumulasikan totalnya dalam perulangan yang sama:

```
for (int i = 0; i < nilaiMhs.length; i++) {
    System.out.print("Masukkan nilai mahasiswa ke-" + (i + 1) + ": ");
    nilaiMhs[i] = input.nextInt();
    total += nilaiMhs[i];
}
```

5. Hitung nilai rata-rata dengan cara nilai total dibagi jumlah elemen dari array nilaiMhs:

```
rata2 = (double) total / nilaiMhs.length;
System.out.println("Total nilai: " + total);
System.out.println("Rata-rata: " + rata2);
```

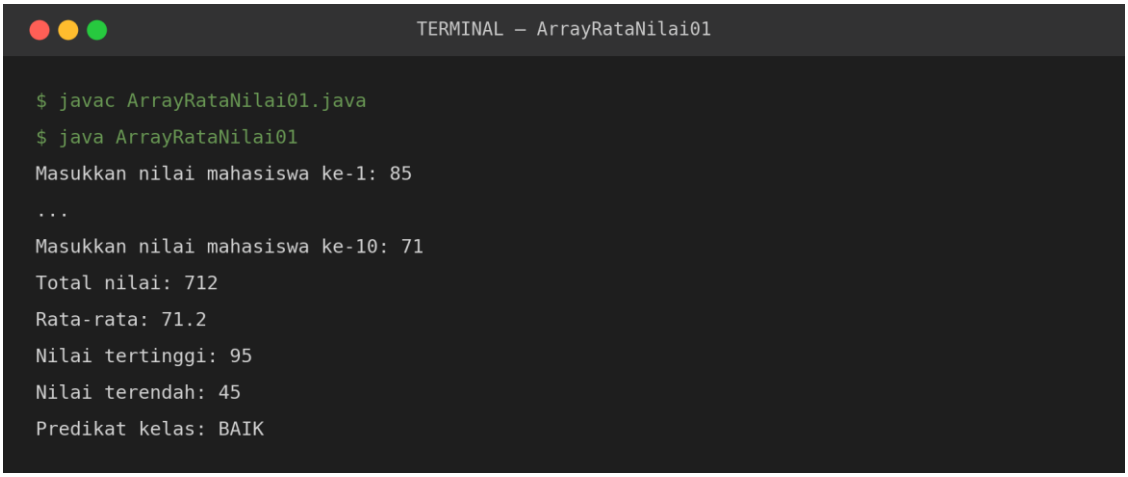
6. Sebagai pengayaan, cari juga nilai tertinggi dan terendah dari array nilaiMhs menggunakan for loop dan variabel pembanding:

```
int tertinggi = nilaiMhs[0];
int terendah = nilaiMhs[0];
for (int i = 1; i < nilaiMhs.length; i++) {
    if (nilaiMhs[i] > tertinggi) tertinggi = nilaiMhs[i];
    if (nilaiMhs[i] < terendah) terendah = nilaiMhs[i];
}
System.out.println("Nilai tertinggi: " + tertinggi);
System.out.println("Nilai terendah: " + terendah);
```

7. Tambahkan kategori predikat kelas berdasarkan nilai rata-rata, misalnya: >= 85 "SANGAT BAIK", >= 70 "BAIK", selebihnya "PERLU PENDAMPINGAN", lalu tampilkan predikat tersebut.
8. Buka terminal VS Code, compile dan jalankan dengan perintah: javac **ArrayRataNilai<NoAbsen>.java** lalu java **ArrayRataNilai<NoAbsen>**. Amati hasilnya dengan beberapa variasi data.

Contoh Hasil Percobaan 3 (Acuan)

Berikut contoh hasil apabila program dijalankan dengan data nilai yang sama seperti Percobaan 2 (85, 60, 72, 90, 45, 78, 66, 95, 50, 71).



Contoh tampilan terminal VS Code setelah program dijalankan dengan data contoh di atas.

Pertanyaan Percobaan 3

1. Modifikasi kode program di atas agar dapat menampilkan banyaknya mahasiswa yang lulus, yaitu mahasiswa yang memiliki nilai lebih besar dari 70 (> 70).
2. Modifikasi program di atas sehingga program menerima jumlah elemen array berdasarkan input dari pengguna (tidak selalu 10), lalu tetap dapat menghitung total, rata-rata, nilai tertinggi, dan nilai terendah dengan benar.
3. Mengapa pada perhitungan rata-rata digunakan (double) total / nilaiMhs.length, bukan langsung total / nilaiMhs.length? Jelaskan!

3.4. Percobaan 4: Pencarian (Searching) dan Pengurutan (Sorting) Sederhana

Waktu Percobaan: 40 menit

Percobaan ini menerapkan algoritma pencarian dan pengurutan sederhana pada data nilai mahasiswa, dua kemampuan dasar yang banyak dipakai dalam pengolahan data nyata.

Bagian A — Linear Search

1. Buka VS Code. Buat file baru, simpan dengan nama **SearchNilai<NoAbsen>.java**
2. Buat array nilai berisi data nilai mahasiswa, lalu tentukan sebuah nilai (*key*) yang ingin dicari:

```
int[] nilai = {78, 65, 90, 55, 88, 72};
int key = 90;
boolean ditemukan = false;

for (int i = 0; i < nilai.length; i++) {
    if (nilai[i] == key) {
        System.out.println("Nilai " + key + " ditemukan pada indeks ke-" + i);
        ditemukan = true;
        break;
    }
}
if (!ditemukan) {
    System.out.println("Nilai " + key + " tidak ditemukan.");
}
```

3. Buka terminal VS Code, jalankan dengan perintah: javac **SearchNilai<NoAbsen>.java** lalu java **SearchNilai<NoAbsen>**. Amati hasilnya, lalu coba ubah nilai key menjadi nilai yang tidak ada di dalam array.

Contoh Hasil Bagian A (Acuan)



Baris pertama menunjukkan hasil pencarian dengan key = 90 (ditemukan). Baris terakhir menunjukkan contoh hasil apabila key diubah menjadi nilai yang tidak ada di dalam array, misalnya 100.

Bagian B — Bubble Sort

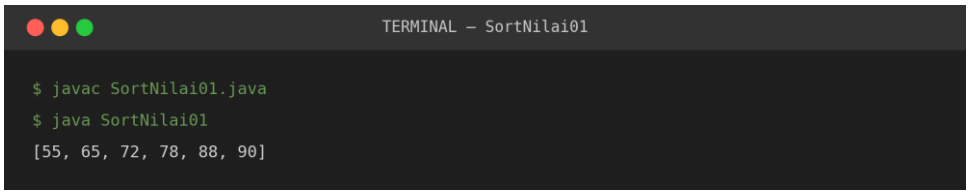
1. Di VS Code, buat file baru, simpan dengan nama **SortNilai<NoAbsen>.java**
2. Gunakan array nilai yang sama seperti Bagian A, lalu urutkan dari nilai terkecil ke terbesar menggunakan algoritma bubble sort:

```
import java.util.Arrays;

int[] nilai = {78, 65, 90, 55, 88, 72};
for (int i = 0; i < nilai.length - 1; i++) {
    for (int j = 0; j < nilai.length - 1 - i; j++) {
        if (nilai[j] > nilai[j + 1]) {
            int temp = nilai[j];
            nilai[j] = nilai[j + 1];
            nilai[j + 1] = temp;
        }
    }
}
System.out.println(Arrays.toString(nilai));
```

3. Buka terminal VS Code, compile dan jalankan dengan perintah: javac **SortNilai<NoAbsen>.java** lalu java **SortNilai<NoAbsen>**. Amati bagaimana urutan array berubah sebelum dan sesudah proses sorting.

Contoh Hasil Bagian B (Acuan)



Array {78, 65, 90, 55, 88, 72} setelah diurutkan dengan bubble sort dari terkecil ke terbesar.

Pertanyaan Percobaan 4

1. Jelaskan maksud dari statement break; pada kode program Bagian A di atas!
2. Modifikasi kode program Bagian A sehingga program dapat menerima input berupa banyaknya elemen array nilai, isi array, dan sebuah nilai (*key*) yang ingin dicari dari pengguna, lalu mencetak indeks posisi elemen dari nilai (*key*) tersebut.
3. Modifikasi program Bagian A agar menampilkan pesan "Nilai yang dicari tidak ditemukan" dengan format pesan yang lebih informatif apabila nilai yang dicari (*key*) tidak ada di dalam array.
4. Jelaskan langkah demi langkah cara kerja algoritma *bubble sort* pada Bagian B menggunakan data ujicoba yang tersedia!
5. Modifikasi program Bagian B agar mengurutkan array dari nilai terbesar ke terkecil (*descending*), bukan dari terkecil ke terbesar.

4. Tugas

Alokasi Waktu: 80 menit (dikerjakan di akhir sesi praktikum, dapat dilanjutkan di luar jam praktikum apabila belum selesai)

4.1. Tugas 1 — Pengelolaan Nilai Mahasiswa

Buatlah program yang dapat menyimpan dan mengelola nilai mahasiswa (bilangan bulat). Program harus menyediakan fitur untuk:

- Memasukkan banyaknya nilai mahasiswa yang akan diinput.
- Memasukkan setiap nilai mahasiswa satu per satu.
- Menghitung nilai rata-rata dari seluruh nilai yang dimasukkan.
- Menampilkan nilai tertinggi dan nilai terendah.
- Menampilkan semua nilai yang telah dimasukkan, beserta status kelulusannya (LULUS / TIDAK LULUS).

4.2. Tugas 2 — Aplikasi Pemesanan Kafe Kampus

Buat program yang dapat mengelola pemesanan makanan dan minuman di kafe kampus. Program menyediakan daftar menu yang sudah ditentukan, memungkinkan pengguna memilih menu melalui fitur pencarian, mencatat pesanan, serta menghitung total biaya.

Daftar menu kafe (inisialisasi array of String saat deklarasi):

```
String[] menu = {"Nasi Goreng", "Mie Goreng", "Roti Bakar", "Kentang Goreng", "Teh Tarik", "Cappucino", "Chocolate Ice"};
int[] harga = {15000, 13000, 10000, 12000, 8000, 18000, 20000};
```

Program yang dibuat wajib mencakup ketentuan berikut:

- Fitur pencarian menu: pengguna memasukkan nama makanan/minuman yang ingin dipesan, program mencari nama tersebut di dalam array menu menggunakan algoritma linear search. Apabila ditemukan, tampilkan harganya; apabila tidak ditemukan, beri tahu pengguna bahwa menu tersebut tidak tersedia.
- Fitur pemesanan: pengguna memasukkan jumlah item yang ingin dipesan, lalu untuk setiap item memilih nama menu (yang harus tervalidasi ada dalam daftar menu). Simpan nama pesanan dalam array satu dimensi terpisah dari array menu utama.
- Hitung dan tampilkan total biaya dari seluruh pesanan yang dimasukkan pengguna.
- Tampilkan daftar pesanan yang telah dibuat beserta total biayanya, dengan format tampilan yang rapi dan mudah dibaca.

Pengumpulan: Kumpulkan kode program (.java) dari Tugas 1 dan Tugas 2, beserta tangkapan layar (screenshot) hasil menjalankan program, kepada dosen/asisten praktikum sesuai ketentuan yang berlaku di kelas masing-masing.